JavaScript is disabled on your browser.

Java SE 27 & JDK 27

Class SelectableChannel

java.lang.Object
    java.nio.channels.spi.AbstractInterruptibleChannel
        java.nio.channels.SelectableChannel

All Implemented Interfaces:

Closeable, AutoCloseable, Channel, InterruptibleChannel

Direct Known Subclasses:

AbstractSelectableChannel

```
public abstract class SelectableChannel
extends AbstractInterruptibleChannel
implements Channel
```

A channel that can be multiplexed via a `Selector`.

In order to be used with a selector, an instance of this class must first be *registered* via the `register` method. This method returns a new `SelectionKey` object that represents the channel's registration with the selector.

Once registered with a selector, a channel remains registered until it is *deregistered*. This involves deallocating whatever resources were allocated to the channel by the selector.

A channel cannot be deregistered directly; instead, the key representing its registration must be *cancelled*. Cancelling a key requests that the channel be deregistered during the selector's next selection operation. A key may be cancelled explicitly by invoking its `cancel` method. All of a channel's keys are cancelled implicitly when the channel is closed, whether by invoking its `close` method or by interrupting a thread blocked in an I/O operation upon the channel.

If the selector itself is closed then the channel will be deregistered, and the key representing its registration will be invalidated, without further delay.

A channel may be registered at most once with any particular selector.

Whether or not a channel is registered with one or more selectors may be determined by invoking the `isRegistered` method.

Selectable channels are safe for use by multiple concurrent threads.

Blocking mode

A selectable channel is either in *blocking* mode or in *non-blocking* mode. In blocking mode, every I/O operation invoked upon the channel will block until it

completes. In non-blocking mode an I/O operation will never block and may transfer fewer bytes than were requested or possibly no bytes at all. The blocking mode of a selectable channel may be determined by invoking its `isBlocking` method.

Newly-created selectable channels are always in blocking mode. Non-blocking mode is most useful in conjunction with selector-based multiplexing. A channel must be placed into non-blocking mode before being registered with a selector, and may not be returned to blocking mode until it has been deregistered.

Since:

1.4

See Also:

`SelectionKey`, `Selector`

Constructor Summary

Constructors

Modifier	Constructor	Description
`protected`	**`SelectableChannel`**`()`	Initializes a new instance of this class.

Method Summary

All Methods | **Instance Methods**

Modifier and Type	Method	Description
`abstract Object`	**`blockingLock`**`()`	Retrieves the object upon which the `configureBlocking` and `register` methods synchronize.
`abstract SelectableChannel`	**`configureBlocking`**`(boolean block)`	Adjusts this channel's blocking mode.
`abstract boolean`	**`isBlocking`**`()`	Tells whether or not every I/O operation on this channel will block until it completes.
`abstract boolean`	**`isRegistered`**`()`	Tells whether or not this channel is currently

		registered with any selectors.
abstract SelectionKey	**keyFor**(Selector sel)	Retrieves the key representing the channel's registration with the given selector.
abstract SelectorProvider	**provider**()	Returns the provider that created this channel.
final SelectionKey	**register**(Selector sel, int ops)	Registers this channel with the given selector, returning a selection key.
abstract SelectionKey	**register**(Selector sel, int ops, Object att)	Registers this channel with the given selector, returning a selection key.
abstract int	**validOps**()	Returns an operation set identifying this channel's supported operations.

Methods declared in class AbstractInterruptibleChannel

begin, close, end, implCloseChannel, isOpen

Methods declared in class Object

clone, equals, finalize, getClass, hashCode, notify, notifyAll, toString, wait, wait, wait

Constructor Details

SelectableChannel

```
protected SelectableChannel()
```

Initializes a new instance of this class.

Method Details

provider

```
public abstract SelectorProvider provider()
```

Returns the provider that created this channel.

Returns:

The provider that created this channel

validOps

```
public abstract int validOps()
```

Returns an operation set identifying this channel's supported operations. The bits that are set in this integer value denote exactly the operations that are valid for this channel. This method always returns the same value for a given concrete channel class.

Returns:

The valid-operation set

isRegistered

```
public abstract boolean isRegistered()
```

Tells whether or not this channel is currently registered with any selectors. A newly-created channel is not registered.

Due to the inherent delay between key cancellation and channel deregistration, a channel may remain registered for some time after all of its keys have been cancelled. A channel may also remain registered for some time after it is closed.

Returns:

`true` if, and only if, this channel is registered

keyFor

```
public abstract SelectionKey keyFor(Selector sel)
```

Retrieves the key representing the channel's registration with the given selector.

Parameters:

`sel` - The selector

Returns:

The key returned when this channel was last registered with the given selector, or `null` if this channel is not currently registered with that selector

register

```
public abstract SelectionKey register(Selector sel,
                                      int ops,
                                      Object att)
                               throws ClosedChannelException
```

Registers this channel with the given selector, returning a selection key.

If this channel is currently registered with the given selector then the selection key representing that registration is returned. The key's interest set will have been changed to `ops`, as if by invoking the `interestOps(int)` method. If the `att` argument is not `null` then the key's attachment will have been set to that value. A `CancelledKeyException` will be thrown if the key has already been cancelled.

Otherwise this channel has not yet been registered with the given selector, so it is registered and the resulting new key is returned. The key's initial interest set will be `ops` and its attachment will be `att`.

This method may be invoked at any time. If this method is invoked while a selection operation is in progress then it has no effect upon that operation; the new registration or change to the key's interest set will be seen by the next selection operation. If this method is invoked while an invocation of `configureBlocking` is in progress then it will block until the channel's blocking mode has been adjusted.

If this channel is closed while this operation is in progress then the key returned by this method will have been cancelled and will therefore be invalid.

Parameters:

`sel` - The selector with which this channel is to be registered

`ops` - The interest set for the resulting key

`att` - The attachment for the resulting key; may be `null`

Returns:

A key representing the registration of this channel with the given selector

Throws:

`ClosedChannelException` - If this channel is closed

`ClosedSelectorException` - If the selector is closed

`IllegalBlockingModeException` - If this channel is in blocking mode

`IllegalSelectorException` - If this channel was not created by the same provider as the given selector

`CancelledKeyException` - If this channel is currently registered with the given selector but the corresponding key has already been cancelled

`IllegalArgumentException` - If a bit in the `ops` set does not correspond to an operation that is supported by this channel, that is, if `set & ~validOps() != 0`

register

```
public final SelectionKey register(Selector sel,
                                   int ops)
                            throws ClosedChannelException
```

Registers this channel with the given selector, returning a selection key.

An invocation of this convenience method of the form

```
sc.register(sel, ops)
```

behaves in exactly the same way as the invocation

```
sc.register(sel, ops, null)
```

Parameters:

`sel` - The selector with which this channel is to be registered

`ops` - The interest set for the resulting key

Returns:

A key representing the registration of this channel with the given selector

Throws:

`ClosedChannelException` - If this channel is closed

`ClosedSelectorException` - If the selector is closed

`IllegalBlockingModeException` - If this channel is in blocking mode

`IllegalSelectorException` - If this channel was not created by the same provider as the given selector

`CancelledKeyException` - If this channel is currently registered with the given selector but the corresponding key has already been cancelled

`IllegalArgumentException` - If a bit in `ops` does not correspond to an operation that is supported by this channel, that is, if `set & ~validOps() != 0`

configureBlocking

```
public abstract SelectableChannel configureBlocking(boolean block)
                                                    throws IOException
```

Adjusts this channel's blocking mode.

If this channel is registered with one or more selectors then an attempt to place it into blocking mode will cause an `IllegalBlockingModeException` to be thrown.

This method may be invoked at any time. The new blocking mode will only affect I/O operations that are initiated after this method returns. For some implementations this may require blocking until all pending I/O operations are complete.

If this method is invoked while another invocation of this method or of the `register` method is in progress then it will first block until the other operation is complete.

Parameters:

`block` - If `true` then this channel will be placed in blocking mode; if `false` then it will be placed non-blocking mode

Returns:

This selectable channel

Throws:

`ClosedChannelException` - If this channel is closed

`IllegalBlockingModeException` - If `block` is `true` and this channel is registered with one or more selectors

`IOException` - If an I/O error occurs

isBlocking

```
public abstract boolean isBlocking()
```

Tells whether or not every I/O operation on this channel will block until it completes. A newly-created channel is always in blocking mode.

If this channel is closed then the value returned by this method is not specified.

Returns:

`true` if, and only if, this channel is in blocking mode

blockingLock

```
public abstract Object blockingLock()
```

Retrieves the object upon which the `configureBlocking` and `register` methods synchronize. This is often useful in the implementation of adaptors that require a specific blocking mode to be maintained for a short period of time.

Returns:

The blocking-mode lock object

Report a bug or suggest an enhancement
For further API reference and developer documentation see the Java SE Documentation, which contains more detailed, developer-targeted descriptions with conceptual overviews, definitions of terms, workarounds, and working code examples. Other versions.
Java is a trademark or registered trademark of Oracle and/or its affiliates in the US and other countries.
Copyright © 1993, 2026, Oracle and/or its affiliates, 500 Oracle Parkway, Redwood Shores, CA 94065 USA.
All rights reserved. Use is subject to license terms and the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.